semantic regions. Regional confidence is updated according to newly observed coverage, enabling the planner to prioritize insufficiently inspected areas and suppress redundant revisits. The updated task state, semantic confidence, and candidate viewpoints are then returned to JTVP for replanning. This closed-loop process continues until all required subtasks are resolved or a task-specific absence condition is established.

IV. METHODOLOGY

The core design philosophy of LogicFly is to leverage the general-purpose reasoning capabilities of VLMs for semantic understanding and task-level decision making, while ensuring stable UAV motion through collision-aware trajectory optimization. This hierarchical integration couples language-grounded task reasoning, reusable semantic–geometric mapping, joint task–viewpoint planning, and safe trajectory execution, enabling reliable long-horizon aerial navigation in complex indoor environments. As illustrated in Fig. 2, the system is composed of four core modules: reusable hierarchical mapping, Conditional Partial-Order Task Graph construction, Joint Task–Viewpoint Planning, and closed-loop perception and execution. This section elaborates on the design principles and implementation details of each module, progressing from reusable representation to task-level planning and finally to online execution and recovery.

A. Reusable 3D Semantic–Geometric Map

The robot first explores an initially unknown environment and constructs a reusable hierarchical semantic–geometric representation from onboard observations. The geometric component $\mathcal{M}_g$ records occupied, observed-free, and unknown 3D space, while the semantic component $\mathcal{M}_s$ organizes floors, rooms, cross-floor transitions, and open-vocabulary object instances in a shared global frame. We use FALCON [36] for autonomous aerial exploration; the resulting trajectories support the acquisition of RGB-D observations, camera poses, and occupancy evidence. The updated perception and room modules below convert these observations into auditable object instances and supported room regions without filling unknown space as free or silently assigning it to a room. This representation supplies semantic anchors for task grounding and geometric free space for navigation. Figure 3 summarizes its construction from exploration data.

1) Target Recognition and Multi-View 3D Fusion: The mapping front end uses text-prompted SAM 3.1 in image mode rather than video tracking. It applies a fixed indoor vocabulary and a confidence threshold specified in Experimental Setup. Each RGB frame returns a 2D proposal, category, confidence, and instance mask. The mask preserves foreground depth only inside its proposal footprint and suppresses box-background depth before calibrated RGB-D pixels are lifted into the world frame. Boxer [37] then estimates an oriented 3D instance

$$\mathbf{o}_{t,k} = (\mathbf{c}_{t,k}, \mathbf{s}_{t,k}, \mathbf{R}_{t,k}, \ell_{t,k}, p_{t,k}), \tag{1}$$

where t is the exact source-frame identity, k is the proposal index, $\mathbf{c}$, $\mathbf{s}$, and $\mathbf{R}$ are the center, size, and orientation, and ℓ and p are the category and confidence.

Multi-view fusion first applies class-consistent non-maximum suppression and then associates proposals using 3D overlap and semantic agreement. Exact capture timestamps are retained as frame identities, and at most the highest-confidence proposal from each frame contributes to one fused instance. The association thresholds and minimum independent-frame requirement are fixed in Experimental Setup. Category names and semantic IDs are jointly tie-broken deterministically. For fused instance j, the confidence-weighted center is

$$\widehat{\mathbf{c}}_j = \frac{\sum_{(t,k)\in\mathcal{I}_j} p_{t,k}\mathbf{c}_{t,k}}{\sum_{(t,k)\in\mathcal{I}_j} p_{t,k}}, \qquad |\{t : (t,k) \in \mathcal{I}_j\}| \geq n_{\text{mv}}. \tag{2}$$

The fused instance becomes the semantic anchor used by room assignment and subsequent language tasks. During online verification, detections enter through the same RGB-D projection interface; known instances use their map IDs, while an unmatched same-category detection becomes a novel online track only after satisfying the minimum-frame and spatial-consistency settings specified in Experimental Setup.

2) Wall- and Observation-Supported Room Partitioning: Room partitioning is performed independently on a horizontal grid for each floor. Wall points extracted from measured geometry are projected into a barrier set $\mathcal{W}$; a bounded local morphological closing only bridges sampling gaps and never hallucinates unknown wall points. The grid resolution and closing neighborhood are fixed in Experimental Setup. Let $\mathcal{G}$ denote the horizontal grid and let $\Omega_0 \subseteq \mathcal{G}$ be the candidate floor domain. To prevent walls or interpolation holes from becoming room evidence, we combine multiple independent support sets: near-horizontal measured surfaces $\mathcal{S}_h$, multi-height free-space observations $\mathcal{S}_f$, multi-frame depth-ray coverage $\mathcal{S}_r$, and recorded trajectory coverage $\mathcal{S}_t$. Each support set is projected onto $\mathcal{G}$ and represented as a set of grid cells. The effective partition domain is

$$\Omega = (\Omega_0 \setminus \mathcal{W}) \cap (\mathcal{S}_h \cup \mathcal{S}_f \cup \mathcal{S}_r \cup \mathcal{S}_t). \tag{3}$$

Thus, Ω contains cells that are not blocked by walls and have support from at least one evidence source. A cell without independent support is excluded from the room partition but is not labeled as exterior or occupied; flight feasibility remains determined by the independent 3D occupancy map.

On Ω, we compute the Euclidean distance transform $d(\mathbf{x})$ to the domain boundary. Broad room interiors have larger clearance than doorways, corridors, and room boundaries, so interior cores are

$$\mathcal{C} = \{\mathbf{x} \in \Omega \mid d(\mathbf{x}) > \tau_c\}, \tag{4}$$

where τ_c is the interior-clearance threshold specified in Experimental Setup. Connected components smaller than A_{min} are discarded, with A_{min} fixed in Experimental Setup. Each retained core is a spatial marker rather than a semantic room label. Marker-controlled watershed propagation runs on $-d$ with grid-connected, multi-source priority-queue flooding and is restricted to Ω; it therefore stops at wall barriers and unsupported boundaries, while low-clearance connections become bottlenecks between competing cores. A small enclosed region is merged into its sole labeled neighbor only when its boundary

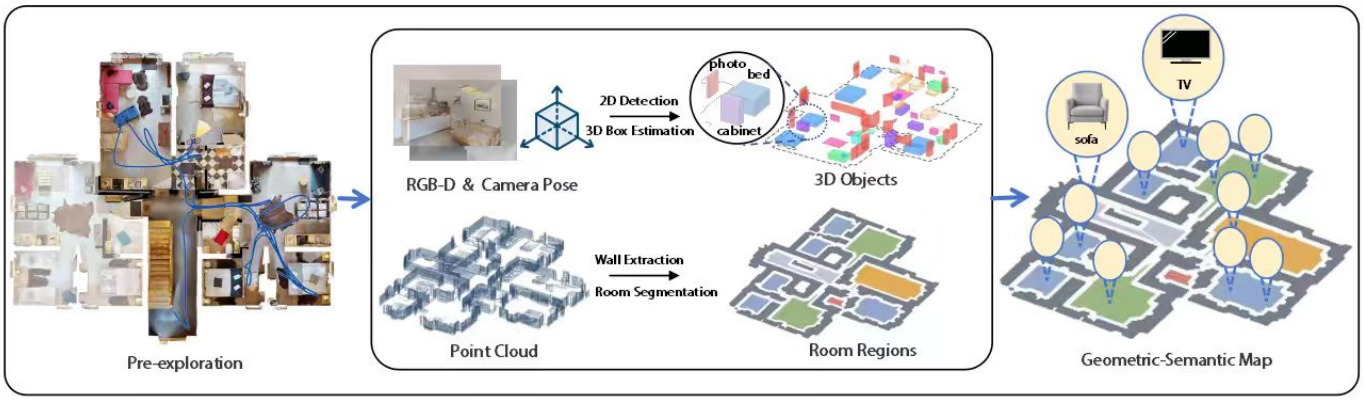


Fig. 3. Hierarchical semantic–geometric representation constructed from exploration data by integrating floor/room geometry and 3D object instances.

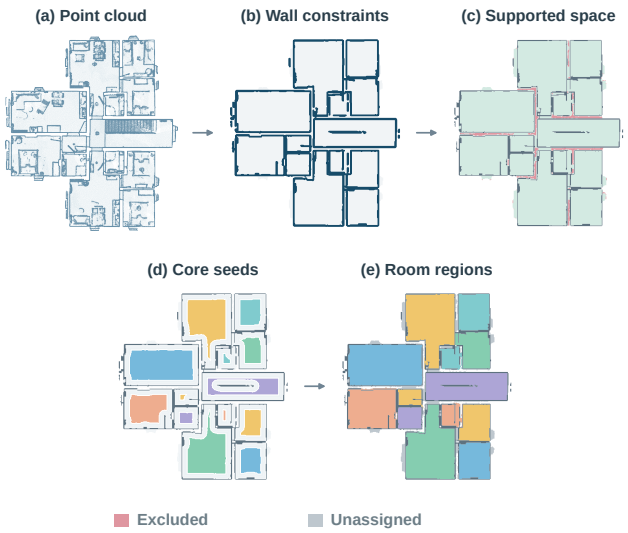


Fig. 4. Room segmentation for hierarchical semantic–geometric mapping. The point cloud is converted into wall-constrained supported space; core seeds are then extracted and expanded into room regions, while excluded and unassigned areas are retained outside the partition.

wall support is weak and it does not border unsupported open space. A supported connected component without a marker remains untyped rather than receiving an artificial seed.

Room categories are assigned only after geometry: measured RGB frames whose camera positions lie in a region are paired with that region's fused 3D objects and passed to a multimodal classifier over a fixed set of room categories. The frame-count setting is given in Experimental Setup. Insufficient evidence yields "unknown", and the classifier cannot change the room boundary or occupancy map. Figure 4 illustrates the resulting wall-constrained supported space, core seeds, and room regions.

B. Conditional Partial-Order Task Graph

Given a natural-language instruction I, the system constructs a **Conditional Partial-Order Task Graph**

$$\mathcal{G}_T = (\mathcal{V}, \mathcal{E}_{\text{prec}}, \mathcal{R}_{\text{cond}}), \tag{5}$$

where $\mathcal{V}$ denotes language-grounded subtasks, $\mathcal{E}_{\text{prec}}$ specifies mandatory precedence relations, and $\mathcal{R}_{\text{cond}}$ contains observation-dependent activation and skip rules. Each task stores its action, semantic target, navigation anchor, verification predicate, spatial constraints, room/floor scope, prerequisites, and search policy. Before execution, the graph is rejected if entity references are invalid, dependencies are inconsistent, or the precedence relation contains a cycle. Figure 5 illustrates the resulting task representation.

For each task, the **navigation anchor** defines the observation region, the **semantic target** specifies the entity to detect, and the **verification predicate** defines the requested evidence. Spatial relations constrain grounding and candidate generation; their satisfaction is recorded geometrically and required by the strict full-task evaluator.

Let $\mathcal{R}_t$ denote tasks whose task-level outcomes have been resolved and $\mathcal{A}_t$ the active pending tasks. The currently executable set is

$$\mathcal{E}_t = \{i \in \mathcal{A}_t : \text{pred}(i) \subseteq \mathcal{R}_t\}. \tag{6}$$

Resolved outcomes are interpreted by $\mathcal{R}_{\text{cond}}$ to activate or skip downstream conditional tasks. Each conditional source is resolved according to its observation policy, either at the first designated viewpoint or after a terminal search outcome. Separating mandatory precedence from conditional activation prevents observation-dependent alternatives from being incorrectly imposed as unconditional order constraints. CPTG thus determines the executable tasks, while JTVP plans their order and observation poses.

C. Task-Conditioned Observation and Motion Modeling

For each executable task i, its navigation anchor and spatial constraints define a task-specific 3D observation region, such as an object, support surface, surrounding area, lower region, or relation-defined area.

The kth candidate observation pose is $q_i^k = (x_i^k, y_i^k, z_i^k, \psi_i^k)$, where (x_i^k, y_i^k, z_i^k) is the camera position and ψ_i^k is yaw. Candidates are sampled at multiple distances, azimuths, and feasible heights, with yaw oriented toward the observation region; vertical visibility is handled by candidate height and the camera field of view.

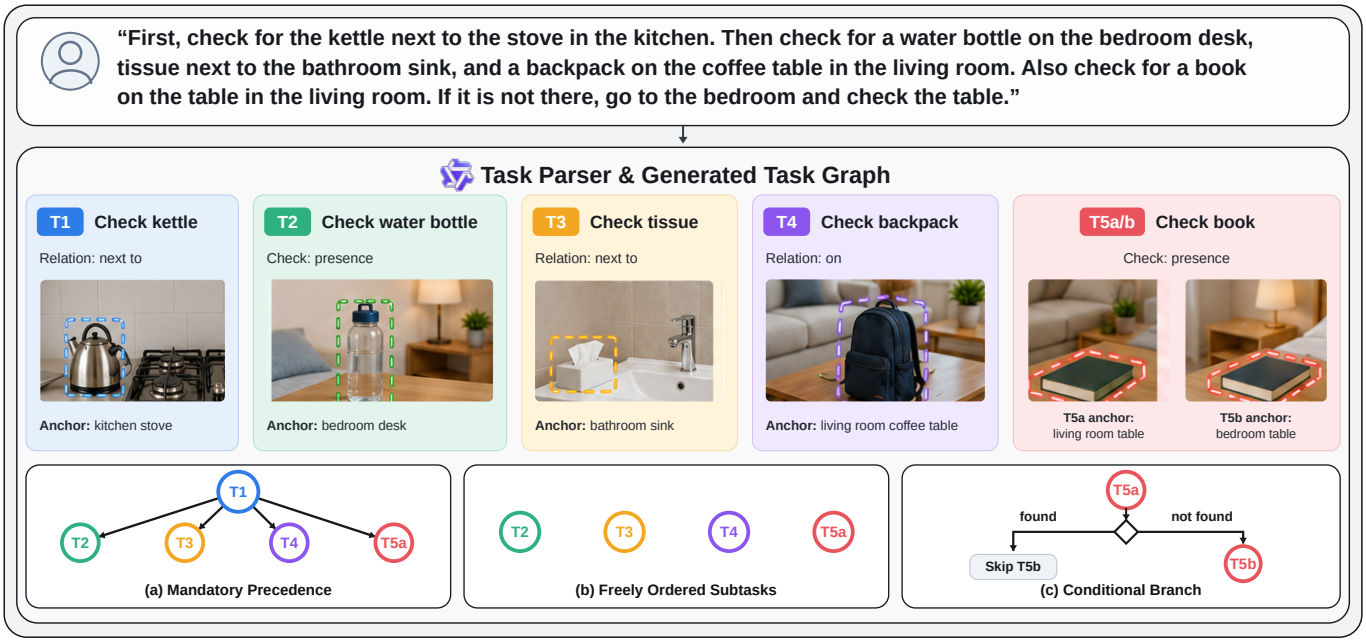


Fig. 5. From a natural-language instruction to a CPTG. The task parser constructs task nodes with navigation anchors, semantic targets, and verification predicates; precedence and conditional edges encode mandatory ordering and observation-dependent activation.

Let $\mathcal{P}_i$ denote samples from the observation region and $\mathcal{V}_i(q) \subseteq \mathcal{P}_i$ the samples visible from q under camera field-of-view and line-of-sight constraints. Geometric coverage is

$$Q_i(q) = \frac{|\mathcal{V}_i(q)|}{\max(1, |\mathcal{P}_i|)}. \tag{7}$$

Candidates violating occupancy, flight-height, spatial-relation, or required-visibility constraints are discarded. The remaining poses are scored by

$$\begin{aligned} C_i^{\text{view}}(q) = \beta_d \frac{|d - d_i^{\text{pref}}|}{\ell_0} + \beta_Q [1 - Q_i(q)] \\ + \beta_z \frac{|z - \bar{z}_i|}{\ell_0} + \beta_r \frac{d_i^{\text{ref}}}{\ell_0} + \beta_b \frac{d_i^{\text{between}}}{\ell_0}. \end{aligned} \tag{8}$$

where d is the sampling distance, d_i^{pref} is the preferred observation distance, $\bar{z}_i$ is the mean observation-region height, d_i^{ref} and d_i^{between} encode task-dependent spatial relations, and ℓ_0 is a reference length. Inapplicable terms are zero. Hard feasibility is enforced before scoring so that a low soft cost cannot compensate for an unsafe or unobservable pose. All weights are nonnegative, and their implementation values are reported in Section V-A. The retained set $\mathcal{Q}_i = \{q_i^1, \ldots, q_i^{K_i}\}$ preserves diverse feasible viewpoints.

Figure 6 illustrates the sampling and feasibility-filtering process.

For candidate poses q_a and q_b, a 3D A* query returns a collision-free polyline $P_t(q_a, q_b) = \{p_1, \ldots, p_N\}$ when the transition is reachable. Let $\Delta\psi_{ab} = \text{wrap}(\psi_b - \psi_a)$. Its length and motion-time proxy are

$$\begin{aligned} L_t(q_a, q_b) &= \sum_{u=1}^{N-1} \|p_{u+1} - p_u\|_2, \\ \widehat{T}_t(q_a, q_b) &= \max\left\{ \frac{L_t(q_a, q_b)}{v_{xy}}, \frac{|z_b - z_a|}{v_z}, \frac{|\Delta\psi_{ab}|}{\omega_\psi} \right\}. \end{aligned} \tag{9}$$

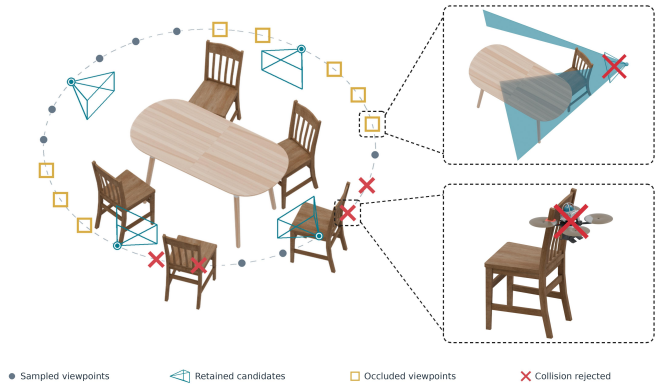


Fig. 6. Task-conditioned viewpoint generation and feasibility filtering. Candidate observation poses are sampled around the target region; occluded or collision-prone poses are rejected, while geometrically feasible candidates with sufficient visibility are retained.

Here, v_{xy} is the nominal translational speed used by the route-length proxy, while v_z and ω_ψ parameterize vertical and yaw motion.

Figure 7 illustrates task-conditioned viewpoint generation and joint selection.

D. Receding-Horizon Joint Task–Viewpoint Planning

At each planning cycle, the planner selects a local task window $\mathcal{W}_t \subseteq \mathcal{E}_t$ with $|\mathcal{W}_t| \leq H$, where H specifies the maximum number of executable tasks jointly considered in the current planning problem.

Candidate poses are grouped by physical location and prioritized using current-pose distance, motion-time estimate,

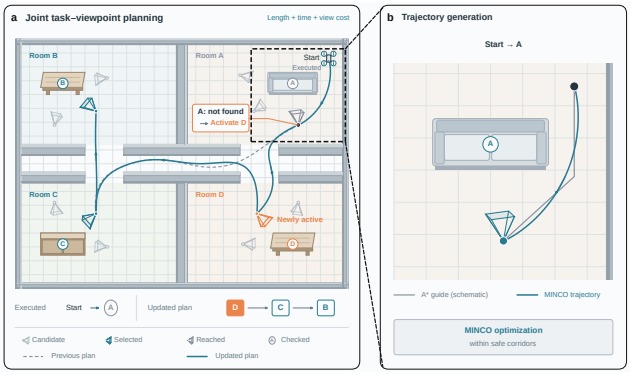


Fig. 7. Task-conditioned viewpoint generation and joint task–viewpoint planning. Candidate observation poses are generated from semantic and spatial constraints, after which JTVP jointly selects task order, observation poses, and the collision-aware route.

and observation score. The retained set $\widetilde{\mathcal{Q}}_{i,t} \subseteq \mathcal{Q}_i$ maintains representative viewpoints across semantic locations and is progressively augmented when alternatives are needed.

Given $m = |\mathcal{W}_t|$, JTVP jointly selects a task ordering $\pi = (\pi_1, \ldots, \pi_m)$ and one observation pose $q_r = q_{\pi_r}^{k_r}$ for each selected task, with $q_0 = q_t^{\text{robot}}$. Let $\iota(q)$ denote the task associated with candidate q. We use the dimensionless transition score

$$c_t(q_a, q_b) = \eta_L \frac{L_t(q_a, q_b)}{\ell_0} + \eta_T \frac{\widehat{T}_t(q_a, q_b)}{\tau_0} + \eta_V C^{\text{view}}_{\iota(q_b)}(q_b), \tag{10}$$

where τ_0 is a reference time. JTVP solves

$$\begin{aligned} \min_{\pi, \mathbf{k}} \quad & \sum_{r=1}^{m} c_t(q_{r-1}, q_r) \\ \text{s.t.} \quad & q_r \in \widetilde{\mathcal{Q}}_{\pi_r, t}, \qquad L_t(q_{r-1}, q_r) < +\infty. \end{aligned} \tag{11}$$

The joint planning problem is solved using subset dynamic programming. Let $D(S, i, k)$ be the minimum accumulated cost of visiting task subset $S \subseteq \mathcal{W}_t$ and ending at candidate k of task i. The initialization, recurrence, and final cost are

$$\begin{aligned} D(\{i\}, i, k) &= c_t(q_0, q_i^k), \\ D(S \cup \{j\}, j, \ell) &= \min_{i \in S, k} \left[D(S, i, k) + c_t(q_i^k, q_j^\ell) \right], \\ J_t^\star &= \min_{i \in \mathcal{W}_t,\, k} D(\mathcal{W}_t, i, k). \end{aligned} \tag{12}$$

Backtracking returns the jointly optimized task order and observation poses. Only the first visit is executed before replanning from the updated robot pose, task state, candidate set, and semantic evidence. This receding-horizon policy preserves the efficiency of multi-task lookahead without committing later visits to assumptions that subsequent observations may invalidate.

E. Safe-Corridor-Constrained Trajectory Optimization

After JTVP selects the next task and observation pose, the trajectory optimizer connects the discrete A* route to the start and goal, constructs geometric safe corridors, and optimizes one continuous trajectory. Safe corridors $\mathcal{K}_r$ are generated from measured point-cloud clearance and CIRI geometric seeds, and the resulting piecewise polynomial trajectory is optimized with MINCO [38]. Unknown voxels are treated as blocked, narrow passages are represented by local guide centerlines and centered half-space constraints, and adjacent corridor intersections are checked to prevent a smoothed curve from cutting across a wall.

Piece r is represented by

$$\begin{aligned} \mathbf{p}_r(\tau) &= \sum_m \mathbf{a}_{r,m} \tau^m, \\ \min_{\mathbf{p}, \mathbf{T}} \quad & J_{\text{smooth}}(\mathbf{p}, \mathbf{T}) + \lambda_c J_{\text{clear}}(\mathbf{p}) \\ & + \lambda_d J_{\text{dyn}}(\mathbf{p}, \mathbf{T}). \end{aligned} \tag{13}$$

subject to endpoint states, inter-piece continuity, corridor membership $\mathbf{p}_r(\tau) \in \mathcal{K}_r$, velocity/acceleration/jerk/snap limits, and terminal yaw. The MINCO objective jointly penalizes smoothness, corridor-clearance deficiency, and dynamic cost, while continuous point-cloud validation remains the final safety authority. Optional height-floor and guide-tracking constraints are auxiliary and cannot override collision checks.

The optimized trajectory is represented by per-piece polynomial coefficients, durations, yaw samples, and the associated collision geometry. Continuous samples are checked for corridor violation, speed/yaw-rate limits, regional height, and raw point-cloud clearance using the collision-clearance setting specified in Experimental Setup. A trajectory is accepted only after these geometric and dynamic checks pass. When replanning is triggered, the already committed prefix is retained and only the remaining route is regenerated.

F. Observation-Driven Semantic Revision and Recovery

At the selected observation pose, RGB observations acquired during motion contribute semantic evidence, while terminal RGB-D observations provide open-vocabulary detections, depth-based 3D projections, and geometric verification cues. A positive online verification advances the task state. A negative terminal observation is handled according to the task-specific not-found policy: it may trigger another local viewpoint, bounded semantic recovery, a terminal absence result, or an observation-dependent branch.

Open-vocabulary detections are lifted into the global frame using depth and camera pose. Same-category observations are associated with mapped instances through spatial consistency, while unassociated detections are accumulated across views for subsequent scene-graph revision.

For tasks configured for semantic recovery, LogicFly distinguishes three levels of negative evidence. A **pose-level miss** removes the attempted candidate while keeping the task active. **Region-level exhaustion** invokes alternative room–functional-region hypotheses grounded in the hierarchy; these hypotheses become geometric observation regions and are ranked using semantic relevance, coverage, and travel cost. A **task-level negative outcome** is produced when the task-specific stopping condition is reached, after which CPTG updates downstream logic. Within semantic recovery, newly covered portions of an unproductive region reduce its belief, shifting search toward alternative hypotheses.

Novel detections are accumulated online through spatially consistent multi-view fusion and materialized in the updated scene graph after execution. Thus, task states and semantic-region beliefs can affect replanning within the current execution, whereas newly created object instances support later use of the revised map.

After each terminal verification, the system updates the task state, candidate poses, and semantic search prior, recomputes $\mathcal{E}_t$, and invokes JTVP from the current robot pose. When a task reaches a task-level outcome, CPTG additionally updates $\mathcal{R}_t$ and evaluates $\mathcal{R}_{\text{cond}}$ to activate or skip downstream conditional tasks. This closes the loop among task logic, 3D observation, geometric motion, and online semantic evidence.

G. Methodology Summary

LogicFly presents a hierarchical perception–reasoning–motion framework for long-horizon aerial navigation. It first builds a semantic–geometric representation that grounds object evidence and room structure in observed geometry. Natural-language instructions are then organized into a Conditional Partial-Order Task Graph (CPTG), enabling precedence constraints and observation-dependent branches to be reasoned about jointly with task-conditioned viewpoints. Within a receding-horizon planning scheme, the system coordinates task selection, viewpoint generation, and collision-aware motion, while safe-corridor-constrained MINCO optimization converts the selected discrete route into a dynamically feasible trajectory. Terminal observations close the loop by verifying task outcomes and updating semantic evidence, candidate viewpoints, and task states. Once a task reaches its terminal outcome, the updated semantic confidence is used to revise the semantic map by incorporating newly verified instances and the corresponding scene-graph updates. Through this integration of semantic task reasoning and geometric motion generation, LogicFly supports reliable execution of complex multi-target instructions in unknown indoor environments.

V. Experiments

We evaluate LogicFly through simulation, real-world UAV experiments, and ablation studies, covering benchmark performance, practical deployment, and the contributions of CPTG and JTVP.

A. Experimental Setup

Evaluation benchmark. We evaluate all methods on six HM3D [39] scenes: 00070, 00166, 00273, 00290, 00337, and 00409. The frozen benchmark contains 152 instruction cases: 61 short single-target cases, 60 single-floor long-instruction cases, and 31 cross-floor long-instruction cases. The schedule uses valid environment seeds and two repetitions per retained start, yielding 854 episodes per method. We remove 54 episode starts that already violate the instruction and evaluate every method on the same clean schedule. The task set covers ordering constraints, conditional execution, semantic recovery, verified absence, spatial relations, and target disambiguation.

Evaluation metrics. We report Trajectory Length (TL), task Success Rate (SR), Success weighted by Path Length (SPL), and Completion Time (CT), together with Navigation Error (NE) when a compatible terminal target position is defined. TL denotes mean executed trajectory length. Except for the explicitly identified OneMap projection, success is defined strictly at the full-task level: every required subtask outcome and realized logical, spatial, disambiguation, and recovery constraint must be satisfied, while skipped conditional branches must not be visited. SPL is computed as

$$\text{SPL}_e = S_e \frac{L_e^{\text{ref}}}{\max(L_e^{\text{ref}}, L_e)}, \tag{14}$$

where S_e is strict binary full-task success, L_e is the executed trajectory length, and L_e^{ref} is the branch- and order-consistent reference route. CT is reported over successful episodes; NE and SPL are N/A for verified-absence tasks without a target position.

Implementation details. Our runner executes CPTGs. The mapping front end uses text-prompted SAM 3.1 image mode with a 30-category indoor vocabulary and detector threshold 0.50. Multi-view fusion uses a 3D IoU threshold of 0.20, an instance-confidence threshold of 0.45, and a minimum of $n_{\text{mv}} = 4$ independent frames; an unmatched online track requires at least two frames whose 3D centers are within $0.75\,\text{m}$. Room partitioning uses a $0.05\,\text{m}$ grid, a 3×3 closing neighborhood, $\tau_c = 0.45\,\text{m}$, $A_{\text{min}} = 0.04\,\text{m}^2$, four-connected grid flooding, and one or two RGB frames for room classification. The JTVP horizon is $H = 3$. Online verification uses the calibrated RGB-D projection interface and open-vocabulary detections; semantic recovery can invoke Qwen3.8-Flash[1] with temperature 0 and thinking disabled. The trajectory optimizer uses a $0.25\,\text{m}$ collision clearance. All planning parameters are fixed across scenes and tasks: $(v_{xy}, v_z, \omega_\psi) = (1.0\ \text{m/s}, 0.5\ \text{m/s}, 1.0\ \text{rad/s})$, $(\ell_0, \tau_0) = (1\ \text{m}, 1\ \text{s})$, $(\beta_d, \beta_Q, \beta_z, \beta_r, \beta_b) = (1, 1, 0.10, 0.15, 0.15)$, and $(\eta_L, \eta_T, \eta_V) = (1, 0.25, 2)$. LogicFly CT is its native task-completion time excluding preparation and post-hoc evaluation, whereas baseline CT is measured from process start to terminal state; all CT values are averaged only over successful episodes. Every process has a 240 s wall-clock limit, and timeouts and algorithm or infrastructure failures remain in the full denominator.

B. Comparison with State-of-the-Art Methods in Simulation

We compare LogicFly with HM3D adaptations of OneMap [22], MapGPT [6], SpatialGPT [7], and SpatialNav [28]. All methods run on the same 854-episode clean schedule. OneMap retains its native ordered-category projection, which omits relations, conditional branches, verified absence, recovery, and return constraints; its SR therefore should not be interpreted as strict full-instruction semantic success. MapGPT, SpatialGPT, and SpatialNav use the frozen common strict evaluator, while LogicFly uses its frozen native strict complete-task evaluator.

As shown in Table I, LogicFly consistently outperforms the compared methods across target count and floor scope. On

[1]Alibaba Cloud Model Studio, model ID `qwen3.8-flash`: https://help.aliyun.com/en/model-studio/qwen3-8-flash (accessed Sep. 15, 2026).